EYAL SEGAL

TECHNICAL HOME ASSIGNMENT

Design Patterns in TypeScript

Six practical Node.js backend exercises | Explanations, examples, tests and trade-offs

Audience: Experienced TypeScript / Node.js backend engineers

Difficulty: Intermediate to advanced

Format: Independent exercises + optional integration challenge

Learning objectives

Understand the intent, appropriate use cases, costs and alternatives for six creational, structural and behavioural design patterns. Each exercise is a simulated backend scenario.

- Create one folder per pattern, with implementation and automated tests.
- Include a README with design rationale, a usage example and alternatives.
- Cover at least one failure scenario and one negative test per pattern.
- Prefer straightforward TypeScript when a pattern would add unnecessary complexity.

Singleton — shared configuration / logger

Creational. Intent: ensure one logical instance exists within a JavaScript runtime. In Node.js, module caching often provides this naturally; a class Singleton is not automatically shared across worker threads, processes or serverless instances.

Example: a Node.js API exposes one AppConfig service. Multiple modules import the same configured instance. A logger is another example, although popular logging libraries already manage logger instances.

Assignment (simulated practice):

- Implement a SettingsProvider in TypeScript with a private constructor and static getInstance().

- Load settings exactly once, freeze the resulting configuration and reject conflicting reinitialisation.
- Provide a test-only reset mechanism; discuss the risks of global mutable state.
- Validation: Use Vitest or Jest to verify reference identity across repeated calls, stable configuration, and test isolation. Explain why Node.js worker threads and clustered processes do not share a singleton.
- Interview discussion: Would exporting a module-level const or using NestJS dependency injection be clearer? What changes in serverless deployments?

Builder — constructing a report request

Creational. Intent: construct a complex object incrementally while separating assembly from the final representation. It is useful for many optional fields or interdependent validation rules.

Example: a report export request has a date range, filters, output format, sorting and optional compression.

Assignment (simulated practice):

- Define an immutable ReportRequest type and a fluent ReportRequestBuilder with withDateRange(), addFilter(), withFormat(), enableCompression(), build().
- Reject inverted date ranges and incompatible combinations at build time.
- Ensure built results do not share mutable arrays with the builder or later builds.
- Validation: Create two distinct reports using the same builder API; test method chaining, validation failures and immutability. Consider readonly and Object.freeze (including nested data).
- Interview discussion: When would a plain object, discriminated union or Zod schema be simpler than a Builder?

Adapter — normalising shipping providers

Structural. Intent: translate an incompatible third-party interface into a stable internal contract, without modifying the external client.

Example: Provider A returns {price:120,currency:"USD"}; Provider B returns {total_amount:12000,currency_code:"USD",minor_units:true}. The application expects RateQuote.

Assignment (simulated practice):

- Define a ShippingRateProvider interface with getQuote(request): Promise<RateQuote>.

- Implement ProviderAAdapter and ProviderBAdapter; normalise money, units and provider-specific errors.
- Keep application code dependent only on ShippingRateProvider.
- Validation: Mock both providers and test equal normalised outputs, invalid currency, timeout and malformed payloads. Use explicit integer minor-unit handling to avoid floating-point errors.
- Interview discussion: Where should retries and currency conversion live? What belongs in an Adapter versus an anti-corruption layer?

Flyweight — shared port metadata

Structural. Intent: share immutable intrinsic state across many logical objects while keeping per-record extrinsic state separate.

Example: millions of shipping-rate records reference the same port metadata (code, country, timezone). Each rate still owns price, timestamp and shipment ID.

Assignment (simulated practice):

- Create a PortMetadataFactory keyed by canonical port code; return the same immutable metadata object for repeated codes.
- Keep mutable rate-specific fields outside the flyweight; do not leak customer data into shared objects.
- Provide a way to measure allocation counts and memory consumption.
- Validation: Assert reference identity for matching port codes, separation for different ports, and unchanged metadata after attempted mutation. Compare heapUsed before/after under controlled Node.js runs (results are illustrative).
- Interview discussion: When can a cache consume more memory than it saves? How would you cap the factory or handle eviction?

Chain of Responsibility — API request checks

Behavioral. Intent: pass a request through ordered handlers; each can reject, handle or forward it to the next step.

Example: an API pipeline checks authentication, request validation, quota and business eligibility before executing a command.

Assignment (simulated practice):

- Define a RequestHandler contract and compose AuthHandler, ValidationHandler, RateLimitHandler and EligibilityHandler.

- Stop the chain on the first rejection and return a typed result; preserve deterministic ordering.
- Support async handlers and ensure a handler does not accidentally call next() twice.
- Validation: Test success, rejection at each stage, order of execution and side effects skipped after rejection. Compare to Express middleware and NestJS guards/interceptors.
- Interview discussion: When is middleware enough? What changes if multiple handlers may independently process the request?

Strategy — retry policies

Behavioral. Intent: encapsulate interchangeable algorithms behind a common interface so the caller is independent of the chosen policy.

Example: outbound HTTP calls use NoRetry, FixedDelayRetry or ExponentialBackoffRetry based on provider reliability and rate limits.

Assignment (simulated practice):

- Define RetryStrategy with shouldRetry(error, attempt) and nextDelayMs(attempt).
- Implement no retry, fixed delay, and exponential backoff with configurable cap and jitter.
- Build an async executeWithRetry<T>() that accepts a strategy and an injected sleep function.
- Validation: Use fake timers or injected sleep to test exact attempts and delays, max-attempt limits, 429 handling and non-retryable errors. Ensure attempt indexing is documented.
- Interview discussion: Why should idempotency influence retry decisions? How do circuit breakers and retry budgets complement Strategy?

Bonus — integrate the six patterns

Build a small TypeScript/Node.js service (Express, Fastify or NestJS) for requesting shipping quotes and exporting reports. Use the patterns only where they reduce real complexity.

- Singleton: application configuration (or explain why DI is preferable).
- Builder: validated report export options.
- Adapter: two external shipping APIs.
- Flyweight: shared port metadata.
- Chain of Responsibility: request checks.
- Strategy: provider-specific retry policy.

- Deliver a README, typed contracts, unit tests, one integration test and a short architecture diagram.

Evaluation rubric

Submission: Node.js 20+, TypeScript 5+, strict mode, Vitest or Jest. Avoid any and prefer discriminated unions or typed errors. These are unofficial educational scenarios, not company interview questions.

- Correctness and edge cases — 30 points.
- TypeScript type design and safe error handling — 20 points.
- Tests and failure scenarios — 20 points.
- Pattern selection and trade-off reasoning — 20 points.
- Code clarity and documentation — 10 points.